NLO Rescue CRO Team

INCHEON RoboCup

RoboCup Junior

Rescue

RVATSKI ROBOTIČKI SAVEZ

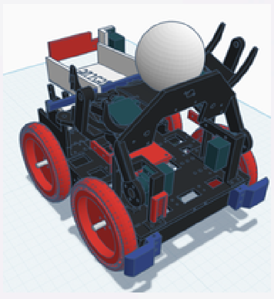
Evolution

At the very beginning, like everything, our robot started with an idea. We began work on the 3D design and parts testing in September 2025, with one clear goal: to build a robot that is reliable, fast and capable of solving all elements of the Rescue Line track without manual intervention. The first version of the robot was built in December 2025. Testing and locating problems on this robot needed to be solved in various ways, such as mechanical adjustments, component or sensor replacements, or even completely changing the concept. We tested the first robot at the National RoboCup Croatia 2026 in March, and that was where the evolution of the robot was really visible. We identified major issues such as excessive weight, suboptimal electronics layout and a PCB that did not fit cleanly inside the chassis. Based on these observations, we designed and built the second robot in March and April 2026. The second version focused on lower weight, better balance, a more compact electronics layout and a smaller, redesigned PCB. We also added the Hailo AI accelerator and reformatted our neural network to the Hailo format, which made victim detection run in real time. Today, when we look back, we can proudly say that every challenge and every effort was worth it. Our final robot is not only the result of technological advancement but also of teamwork, creativity and dedication of the entire team.

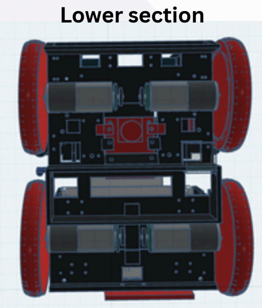
CAD Design

To facilitate easy adjustments and ease of repair later in the project, all structural components were chosen to be 3D printed. This method allows precise control over the shape and size of the components. We designed the chassis in Fusion 360 and Tinkercad, and built two versions during testing. The first version was a critical role in defining the layout. During testing, errors in design were identified, particularly parts with thin structures that tended to break easily. These flawed designs were then iteratively redesigned and reprinted throughout the development cycle, ensuring robustness and reliability in the final product. The robot is split into two main sections – the lower section holds the motors, the battery and the line camera with its lighting diodes, while the upper section holds the PCB, the Raspberry Pi, the wiring and the evacuation zone camera. This layered structure improves accessibility and makes it possible to change parts quickly when something breaks during testing.

Robot in Thinkercad



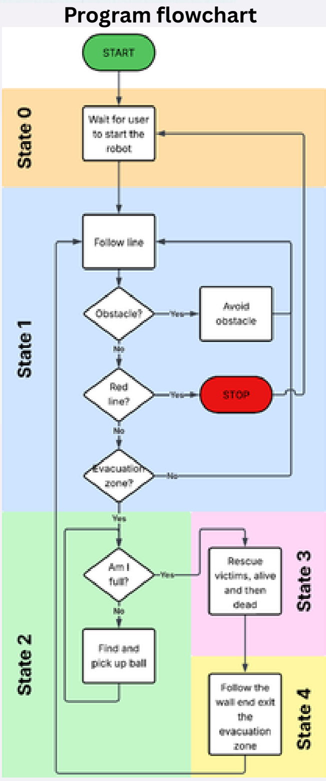
Lower section



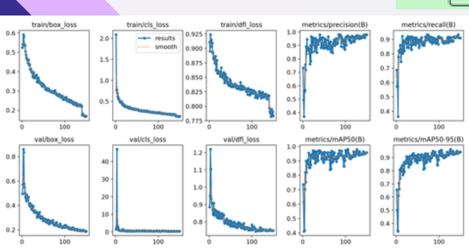
Computer Vision

We trained a custom neural network for victim detection on around 2000 images that we collected ourselves on different Rescue Line evacuation zones. The images were labelled in Roboflow and the model was trained on a local GPU using the Ultralytics framework. We chose a small model architecture so that it can run fast enough on the Raspberry Pi together with the Hailo accelerator. The neural network detects the live victims, the dead victim and the collection (drop) areas. After positioning itself in the centre of the evacuation zone, the robot starts spinning slowly and searches for victims. The robot first finds the two live victims and rescues them together, and only after that it searches for the dead victim. Used together with the Hailo AI HAT, the model runs in real time while the robot is moving, which makes the system more responsive and improves detection of victims close to walls – the hardest case in our earlier tests.

Program flowchart



Progres of the NN model in training



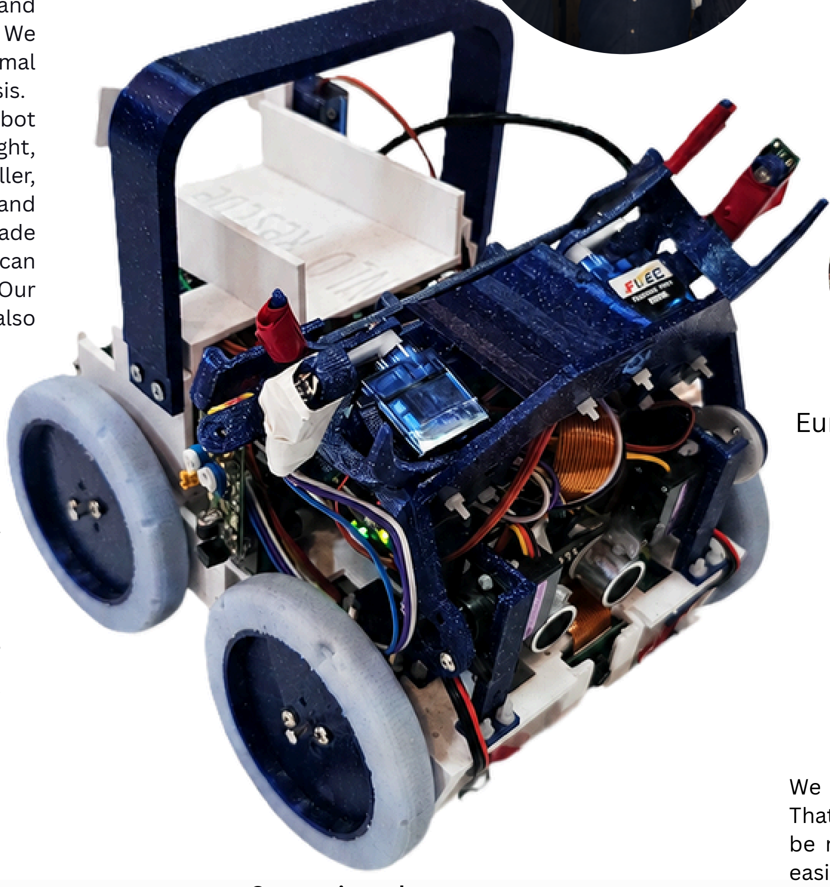
Sensors

Line CAM

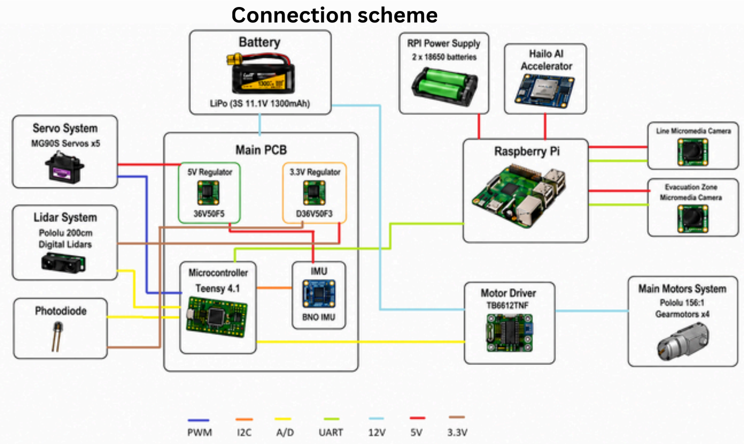
The line camera is used for detecting the black line, intersections, green markers and the red line. It works together with our custom diode lighting, which keeps the image stable in different rooms and lighting conditions.

Evacuation Zone CAM

The evacuation zone camera is used when the robot enters the evacuation zone. It gives the Raspberry Pi a wide view of the zone, so the neural network can detect live victims, dead victims and drop areas.



Connection scheme



Main program loop

The robot uses a software architecture split between two systems. The Teensy 4.1 runs Arduino C++ code and handles all low-level control: motor driving, PID, reading the IMU and sensors and controlling the servos. The Raspberry Pi 5 runs Python with OpenCV for image processing and the Hailo AI accelerator for the neural network. The two systems communicate via UART – the Raspberry Pi sends the line position, angle, intersection type and victim positions, and the Teensy uses this information together with its own sensor data to decide how to move at every moment. The whole program runs in five states. In State 1 the robot follows the line. The Raspberry Pi reads the line camera, applies a colour threshold to isolate the black line, finds its contour and calculates the line centre position and angle. These are sent to the Teensy, which runs a PID controller that steers the robot back to the centre smoothly. A custom diode lighting ring around the line camera keeps the image consistent in any room. While following the line, the Raspberry Pi also checks for green markers for intersection turns, the Teensy monitors the ultrasonic sensor for obstacles and the IMU for ramps, and automatically adjusts speed and direction when needed. When the silver line is detected, the robot stops and moves to State 2. In State 2 the robot enters the evacuation zone, positions itself in the centre and starts spinning while the neural network searches for victims. When a victim is found, the robot drives towards it and picks it up with the arm – a photodiode on the arm confirms the victim is captured. Next it enters State 3, in which the robot first rescues the two live victims and places them in the live victim area, then searches for the dead victim (state 2) and drops it in the dead victim area. Once all victims are delivered, the robot enters State 4 and uses the IR sensors to follow the wall until it finds the exit and leaves the evacuation zone.

IMU

The IMU is used for precise turns, orientation and ramp detection. It helps the robot know when it is climbing a ramp, when it changes angle, and when it needs to recover from slipping or unstable movement.

Lidars

The lidars are used to measure distance from walls and objects. They help the robot navigate inside the evacuation zone, detect obstacles and keep the correct distance while moving.

Switches

The switches are used as simple contact sensors and control inputs. They can be used to start the robot or detect when a mechanical part has reached a certain position.

Photodiode

The photodiode is mounted on the victim handling arm. It is used to confirm that a victim has been successfully picked up by the robot.

Team members

Sven Ridzak (left) -Sven leads the team and is responsible for the software that runs inside the evacuation zone, including victim detection, victim handling and exit search. He also designed and built the custom made PCB and worked on the overall hardware integration.

Jan Ridzak (right) - Jan is responsible for the line following software, including image processing of the line camera, intersection handling and red line detection. He also designs all 3D printed parts of the robot in Fusion 360, including the chassis, wheels and mounts.

Mentors

Juraj Kolarić (middle) and Ivica Kolarić (far left)

Achievments

RoboCup Croatia 2026- 1st place

European RoboCup Austria 2026- 3rd place

Innovations

Custom debris removing tools

We expected a lot of debris like shredded papers, chopsticks... That is why we developed custom debris removing tools that can be mounted on the front of the robot to make line detection easier and to remove slipping effects caused by the debris. We developed two custom tools, one specialised for chopsticks and the other for any type of sponges or shredded paper. The most important thing was that they still allowed the robot to pass bumps and ramps without any problem. We did that by integrating springs in our tool so it can bend but only for objects glued to the track.

Custom silicone-coated wheels

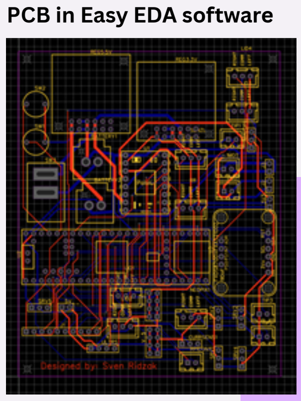
To get the best possible grip we designed custom wheels with a 3D printed core and a silicone outer layer. The silicone provides high grip on the field surface, shock absorption when crossing intersections and bumps, better performance on ramps and the option to customize the wheel exactly for our robot. We tested several silicone hardness values before picking the one that gave the best compromise between grip and rolling resistance. This is one of the main reasons our second robot now passes ramps with bumps much better then the first.

Debris removing tools and silicon wheels

PCB Design

To accommodate various components and connectors, we designed a custom PCB in EasyEDA. The PCB minimizes the number of cables, resulting in a more compact and organized design, while also efficiently managing power distribution. Component placement was carefully planned so that signal routing is short and power and signal lines do not cross unnecessarily. We developed two versions of the PCB. The first version was too large and did not fit optimally inside the chassis. As the robot changed shape, the mechanical and hardware requirements changed too, and so did the structure and form of the PCB itself. The second version was redesigned to be significantly smaller and more efficient, which improved overall reliability and made the whole robot lighter. JST XH connectors and standard headers are used for stable connections – the power converters, motor drivers and cameras are directly mounted on the PCB using specific sockets to interface with the Teensy, the Raspberry Pi and the sensors, ensuring straightforward and secure connections.

PCB in Easy EDA software



Soldered PCB